

Introducción a la placa de desarrollo del curso y a VSCode + ESP-IDF plugin



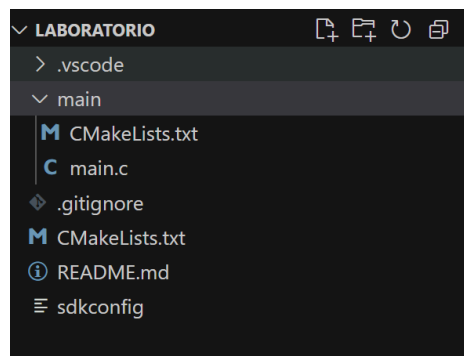
Agustina Bacigalupe - Pierina Borsieri - Sofia Nicoletti - Adrián Tesore

Universidad Católica del Uruguay

Mayo 2025

Montevideo, Uruguay

Parte 1

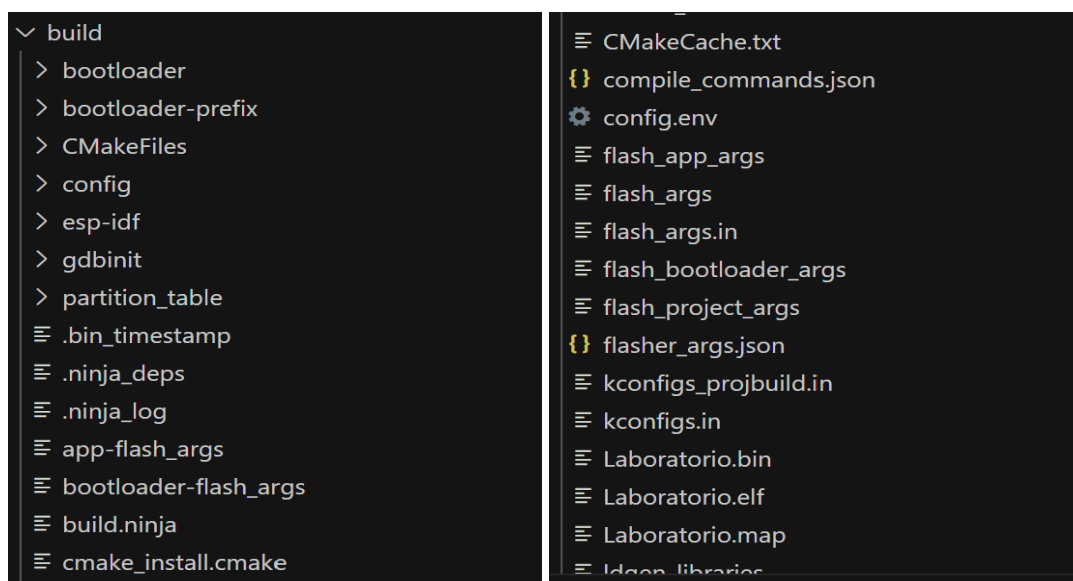


Información del compilador :

Memory Type/Section	Used [bytes]	Used [%]	Remain [bytes]	Total [bytes]
Flash Code	82118	1.04	7782170	7864288
.text	82118	1.04		
DIRAM	57119	33.2	114913	172032
.text	46948	27.29		
.data	7064	4.11		
.bss	2080	1.21		
.vectors	1027	0.6		
Flash Data	36776	0.89	4091960	4128736
.rodata	36520	0.88		
.appdesc	256	0.01		
RTC FAST	68	0.83	8124	8192
.force_fast	28	0.34		
.rtc_reserved	24	0.29		

Total image size: 173961 bytes (.bin may be padded larger)

Se genera el build del proyecto que tiene los ejecutable:



2) Parámetros de configuración:

Cada vez que se modifica una configuración del proyecto (por ejemplo, el nivel de log, el uso de Wi-Fi o la frecuencia del procesador), dichos cambios se guardan en el archivo `sdkconfig`. Si se guarda una nueva configuración, la versión anterior se guarda automáticamente en otro archivo llamado `sdkconfig.old`.

Esto permite comparar los cambios que se fueron haciendo y, si es necesario, volver a lo anterior de forma más sencilla. De esta manera es que se nos permite controlar las configuraciones del proyecto.

3) Compilación:

En el primer archivo `flasher_args.json` se puede observar que contiene, las direcciones de memoria donde se debe grabar cada binario, la ruta de los archivos `.bin`, opciones de configuración como la velocidad de transferencia por puerto serie.

En el segundo llamado `project_description.json` el nombre del proyecto, el tipo de microcontrolador utilizado `esp32`, la versión del framework `ESP-IDF`, las rutas de los archivos fuente (`main.c`, etc.), información sobre dependencias del proyecto.

Ambos archivos son generados automáticamente por el sistema.

exampleData: Tipo: `int`, Tamaño: 4 bytes, Dirección de memoria: `0x3ffc1c90`

exampleArray: Tipo: `char[12]`, Tamaño: 12 bytes, Dirección de memoria: `0x3ffc1c84`

Ambas variables están ubicadas en la sección de RAM, en direcciones contiguas. Se observa cómo el tamaño y la alineación afectan las direcciones asignadas en memoria.

Al sustituir `ARRAY_SIZE` por `10`, vemos que la dirección de memoria sigue siendo la misma: `0x3ffc1c84`, pero en cambio el tamaño del arreglo cambió, siendo así que pasó de 12 bytes (`0xC`) a 10 bytes (`0xA`).

Parte 2

Para controlar dicho componente luego de ver previamente el datasheet sabiendo que se le envía una secuencia de 24 bits: 8 bits para el color verde, 8 bits para el color rojo, 8 bits para el color azul, entonces, cada bit se transmite con una señal de pulso que depende del tiempo: Un "1" lógico es un pulso alto largo y un bajo corto y un "0" lógico es un pulso alto corto y un bajo largo.

Entonces creamos el siguiente pseudocódigo:

Comienzo

Inicializamos el pin de datos como salida

Para cada color en orden (verde, rojo, azul),

Para cada bit del color (de mayor a menor),

Si el bit es 1 entonces

 Generar pulso alto largo

 Generar pulso bajo corto

fin si

Si el bit es 0 entonces

 Generar pulso alto corto

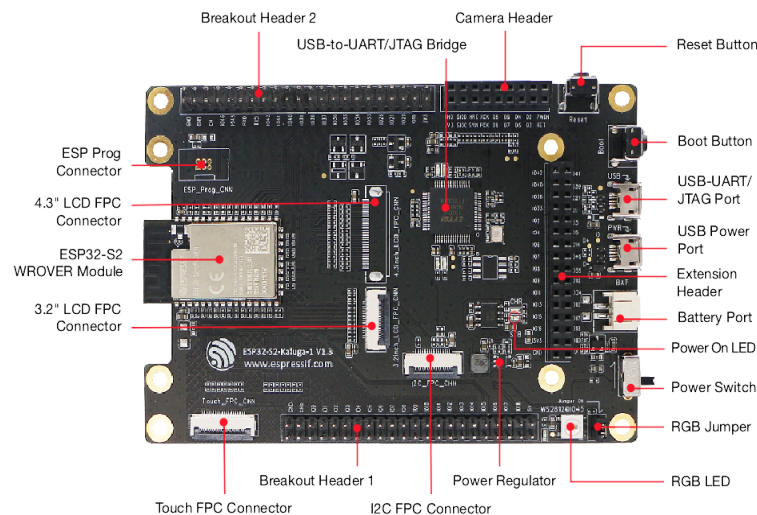
 Generar pulso bajo largo

fin si

Fin

Hardware:

Para poder controlar el LED RGB embebido en la placa ESP32-S2-Kaluga-1, es necesario realizar una conexión física mediante un jumper en el lugar correcto de la placa. De acuerdo con la guía oficial de usuario del kit ESP32-S2-Kaluga-1, el LED RGB (modelo WS2812C) está conectado a la placa mediante un pin de entrada de datos llamado RGB_CTRL. Este pin no está conectado por defecto al microcontrolador, por lo que es necesario colocar un jumper en el conector J4, que une el pin EXT_GPIO45 (correspondiente al GPIO45 del ESP32-S2) con la entrada RGB_CTRL del LED. Esta conexión permite que el microcontrolador pueda enviar datos al LED para controlar su color.

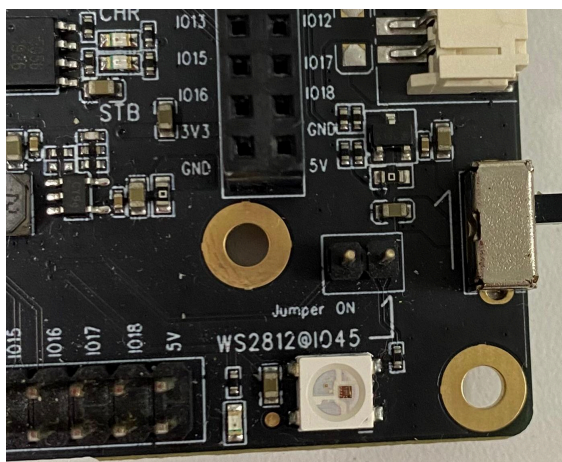


Ubicación del jumper J4:

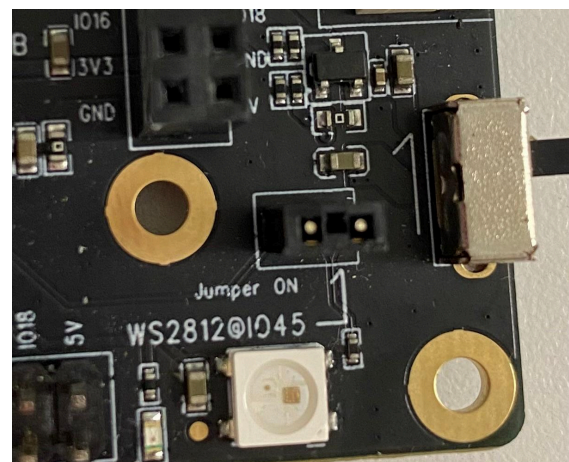
- Se ubicó el conector J4 en la placa, según el diagrama de la documentación oficial.
- Este conector está rotulado como J4 y se encuentra en el borde de la placa cerca del LED RGB.

Colocación del jumper:

- Se conectó un jumper entre los pines del conector J4 para establecer la conexión entre GPIO45 (EXT_GPIO45) y la entrada de datos RGB_CTRL del LED



sin jumper



con jumper

Explicación del código

Se comenzó creando una carpeta “components” para luego preceder en ella a agregar tres archivos. Uno de estos, llamado led_strip, el cual fue donde se incluyó la librería brindada por el docente. Continuando por las dos restantes que fueron de delay y led, el primero, encargado de utilizar la función nativa delay del ESP32- s2 “esp_rom_delay_us”, el mismo, originalmente toma valores en microsegundos, lo que debió ser modificado para el óptimo funcionamiento, de esta forma se cambió de micro a milisegundos. Finalizando, en el archivo restante, se implementó el código que genera el parpadeo del led, para ello se aplicaron las funciones de la librería anteriormente mencionada led_strip. La función led_off, fue creada con el fin de apagar el led, por otro parte, led_init es quien inicializa el led, con estas tres

funciones es que se logra el parpadeo del led: led_init, delay_ms, led_off, actuando respectivamente.

Ademas, se nos pidió el cambio de color en el led, para esto agregamos la funcion led_set_color, que por medio de set_pixel que fue la funcion que se nos proporciono, se setean los distintos colores en rgb. Por último, como se puede ver en la imagen, se creó el loop donde setea el color, se espera un determinado tiempo utilizando la función de delay_ms, se apaga el led y se vuelve a esperar un intervalo de tiempo para volver a encenderlo, actuando así para cada color.

```
void led_blink_colors_loop(led_strip_t *strip)
{
    while (1)
    {
        led_set_color(strip, 255, 0, 0); // rojo
        delay_ms(500);
        led_off(strip);
        delay_ms(500);

        led_set_color(strip, 0, 255, 0); // verde
        delay_ms(500);
        led_off(strip);
        delay_ms(500);

        led_set_color(strip, 0, 0, 255); // azul
        delay_ms(500);
        led_off(strip);
        delay_ms(500);
    }
}
```

Conclusión:

En este laboratorio pudimos compilar y correr un proyecto usando la placa ESP32-S2-Kaluga-1, entendiendo cómo está organizada su estructura y cómo interactuar con sus periféricos. Para poder controlar el LED RGB embebido, fue necesario colocar un jumper en el conector J4, lo que conecta el GPIO45 con la entrada del LED (RGB_CTRL).

Aprendimos cómo funciona el LED WS2812C, y usamos una librería que ya venía hecha con el driver RMT para controlarlo. A partir de ahí, hicimos una librería propia

(led.h y delay.h) para encapsular la lógica de control del LED, incluyendo funciones para prenderlo, apagarlo y hacerlo parpadear en distintos colores. Además, hicimos una librería separada para manejar los delays.

Se logró que el LED terminara funcionando como queríamos, cambiando de color en bucle.